

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки 09.03.04 «Программная инженерия» –

Системное и прикладное программное обеспечение

Отчёт

По лабораторной работе №3

по предмету

Распределенные системы хранения данных

Вариант: 75623

Выполнили:

студенты 3 курса

Батманов Даниил Евгеньевич

Демидов Иван Алексеевич

Группа: Р3307

Принял:

Николаев Владимир Вячеславович

Отчёт принят «__»_____2025 г.

Оценка: _____

г. Санкт-Петербург, 2025

Задание:

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Требования к отчёту

Отчет должен быть самостоятельным документом (без ссылок на внешние ресурсы), содержать всю последовательность команд и исходный код скриптов по каждому пункту задания. Для демонстрации результатов приводить команду вместе с выводом (самой наглядной частью вывода, при необходимости).

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:
Первоначальная полная копия + непрерывное архивирование.
Включить для СУБД режим архивирования WAL; настроить копирование WAL (scp) на резервный узел; создать первоначальную резервную копию (pg_basebackup), скопировать на резервный узел (rsync).
- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
 - Средний объем новых данных в БД за сутки: **800МБ**.
 - Средний объем измененных данных за сутки: **800МБ**.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
 - удалить с диска директорию любой таблицы со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.
- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
 - исходное расположение директории PGDATA недоступно - разместить данные в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Генерация файла на резервном узле с помощью pg_dump и последующее применение файла на основном узле.

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
 - удалить любые две таблицы (DROP TABLE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

Выполнение:

Кредсы:

helios: s367868@helios.cs.ifmo.ru ivMp,1936

lab2: виртуалка pg105, пользователь postgres1, 3kJ.t4v4

lab3: виртуалка pg106, пользователь postgres3, 5r1yxEcM
securepass

Этап 1: Резервное копирование

Настройка архивирования WAL

1. В файл *postgresql.conf* добавлены параметры:

wal_level = replica

archive_mode = on

archive_command = 'scp %p

postgres3@pg106:/var/db/postgres3/wal_archive/%f

2. В файл *pg_hba.conf* добавлены правила репликации:

local	replication	all		peer
host	replication	all	127.0.0.1/32	password
host	replication	all	:::1/128	password
host	replication	postgres1	192.168.11.105/32	password

3. Перезапуск PostgreSQL:

pg_ctl restart -D /var/db/postgres1/cax22

Создание полной резервной копии

Скрипт backup.sh:

#!/bin/bash

Очистка старых резервных копий

rm -rf /var/db/postgres1/tablespace_backup

mkdir -p /var/db/postgres1/tablespace_backup/cei80

mkdir -p /var/db/postgres1/tablespace_backup/jax56

mkdir -p /var/db/postgres1/tablespace_backup/jer28

rm -rf /var/db/postgres1/lab3_backup

mkdir /var/db/postgres1/lab3_backup

```
# Создаем базовую резервную копию с указанием табличных пространств
pg_basebackup -D /var/db/postgres1/lab3_backup -T
/var/db/postgres1/cei80=/var/db/postgres1/tablespace_backup/cei80 -T
/var/db/postgres1/jax56=/var/db/postgres1/tablespace_backup/jax56 -T
/var/db/postgres1/jer28=/var/db/postgres1/tablespace_backup/jer28 -X stream
--progress --verbose -p 9425 --checkpoint=fast -U postgres1
```

```
# Копируем на резервный узел (перезаписываем целевые каталоги)
rsync -av --delete /var/db/postgres1/tablespace_backup/cei80/
postgres3@pg106:/var/db/postgres3/cei80/
rsync -av --delete /var/db/postgres1/tablespace_backup/jax56/
postgres3@pg106:/var/db/postgres3/jax56/
rsync -av --delete /var/db/postgres1/tablespace_backup/jer28/
postgres3@pg106:/var/db/postgres3/jer28/
```

```
# Копируем основную резервную копию
rsync -av --delete /var/db/postgres1/lab3_backup/
postgres3@pg106:/var/db/postgres3/cax22/
```

```
rsync -av --delete /var/db/postgres1/lab3_backup/pg_wal/
postgres3@pg106:/var/db/postgres3/ems92/
```

```
[postgres1@pg105 ~]$ pg_basebackup \
-D /tmp/lab3_backup \
-h 192.168.11.105 \
-p 9425 \
-U postgres1 \
-X fetch \
--checkpoint=fast \
-P \
-T /var/db/postgres1/cei80=/tmp/lab3_backup_cei80 \
-T /var/db/postgres1/jer28=/tmp/lab3_backup_jer28 \
-T /var/db/postgres1/jax56=/tmp/lab3_backup_jax56
Пароль:
ПРЕДУПРЕЖДЕНИЕ: специальный файл "./.s.PGSQL.9425" пропускается
ПРЕДУПРЕЖДЕНИЕ: специальный файл "./.s.PGSQL.9425" пропускается
55254/55254 КБ (100%), табличное пространство 4/4
```

```

pg_snapshots/
pg_stat/
pg_stat_tmp/
pg_subtrans/
pg_tblspc/
pg_twophase/
pg_wal/
pg_wal/0000000100000000000000026
pg_wal/archive_status/
pg_wal/archive_status/0000000100000000000000026.done
pg_xact/
pg_xact/0000

sent 74.148 bytes received 310.382 bytes 40.476,84 bytes/sec
total size is 55.708.151 speedup is 144,87
[postgres1@pg105 ~]$ █

```

Расчет объема резервных копий

- Новые данные за месяц: 800 МБ/день * 30 = 24 ГБ
- Измененные данные (WAL): 800 МБ/день * 30 = 24 ГБ

Итого: 48 ГБ + объем полной копии

Анализ: Использование непрерывного архивирования WAL позволяет минимизировать потери данных, но требует хранения большого объёма журнальных файлов.

Этап 2: Восстановление на резервном узле

Скрипт восстановления `restore.sh`:

```
#!/bin/bash
```

```
# Заменяем путь сокетов на резервном узле
```

```
sed -i "s|unix_socket_directories =
```

```
'/var/db/postgres1/cax22'|unix_socket_directories = '/var/db/postgres3/cax22'|g"
```

```
/var/db/postgres3/cax22/postgresql.conf
```

```
# Настраиваем восстановление
```

```
echo "restore_command = 'cp /var/db/postgres3/ems92/%f%p'" >>
```

```
/var/db/postgres3/cax22/postgresql.conf
```

```
# Создаем сигнальный файл для восстановления
```

```
touch /var/db/postgres3/cax22/recovery.signal
```

Настраиваем права

```
chmod 700 /var/db/postgres3/cax22/
```

```
rm -f /var/db/postgres3/cax22/pg_tblspc/*
```

Создаем симлинки для табличных пространств

```
ln -s /var/db/postgres3/cei80 /var/db/postgres3/cax22/pg_tblspc/16388
```

```
ln -s /var/db/postgres3/jax56 /var/db/postgres3/cax22/pg_tblspc/16389
```

```
ln -s /var/db/postgres3/jer28 /var/db/postgres3/cax22/pg_tblspc/16390
```

Запускаем PostgreSQL

```
pg_ctl start -l /var/db/postgres3/pg_ctl_logfile.log -D /var/db/postgres3/cax22
```

Проверка восстановления:

```
psql -U postgres3 -p 9425 -d goodgoldlake -c "SELECT * FROM sales111;"
```

Этап 3: Восстановление после повреждения файлов

Симуляция сбоя:

```
rm -rf
```

```
/var/db/postgres1/cax22/pg_tblspc/16388/Pg_16_202307071/16391/16398
```

Удаление директории таблицы

Пример сбоя:

```
[postgres1@pg105 ~]$ psql -p 9425 -U postgres1 -d goodgoldlake
psql (16.4)
Введите "help", чтобы получить справку.

goodgoldlake=# select * from sales111;
ОШИБКА: не удалось открыть файл "pg_tblspc/16388/Pg_16_202307071/16391/16398": No such file or directory
goodgoldlake=# select * from sales1sdfsdf;
ОШИБКА: отношение "sales1sdfsdf" не существует
СТРОКА 1: select * from sales1sdfsdf;
```

Восстановление из резервной копии:

```
#!/bin/bash
```

```
rm -rf /var/db/postgres1/new_db
```

```
mkdir new_db
```

```
scp -r postgres3@pg106:/var/db/postgres3/cax22 /var/db/postgres1/new_db/
```

```
scp -r postgres3@pg106:/var/db/postgres3/cei80 /var/db/postgres1/new_db/
```

```
scp -r postgres3@pg106:/var/db/postgres3/jax56 /var/db/postgres1/new_db/
```

```
scp -r postgres3@pg106:/var/db/postgres3/jer28 /var/db/postgres1/new_db/  
scp -r postgres3@pg106:/var/db/postgres3/ems92 /var/db/postgres1/new_db/
```

```
# Исправить конфигурацию
```

```
sed -i "s|unix_socket_directories =  
'/var/db/postgres3/cax22'|unix_socket_directories =  
'/var/db/postgres1/new_db/cax22'|g"  
/var/db/postgres1/new_db/cax22/postgresql.conf  
sed -i "s|restore_command = 'cp /var/db/postgres3/ems92/%f  
%p'|restore_command = 'cp /var/db/postgres1/new_db/ems92/%f %p'|g"  
/var/db/postgres1/new_db/cax22/postgresql.conf
```

```
# Удалить старые симлинки и создать новые
```

```
rm -rf /var/db/postgres1/new_db/cax22/pg_tblspc/16388  
rm -rf /var/db/postgres1/new_db/cax22/pg_tblspc/16389  
rm -rf /var/db/postgres1/new_db/cax22/pg_tblspc/16390  
ln -s /var/db/postgres1/new_db/cej80  
/var/db/postgres1/new_db/cax22/pg_tblspc/16388  
ln -s /var/db/postgres1/new_db/jax56  
/var/db/postgres1/new_db/cax22/pg_tblspc/16389  
ln -s /var/db/postgres1/new_db/jer28  
/var/db/postgres1/new_db/cax22/pg_tblspc/16390
```

```
# Запустить восстановление
```

```
touch /var/db/postgres1/new_db/cax22/recovery.signal
```

Этап 4: Восстановление после логического повреждения


```
goodgoldlake=# select * from logs111 offset 995;
```

id	entry
996	8c77ae1534a4ac4544404589693f3da1
997	0bb6a25c8d5e6753cf3267db7ea58fd5
998	98356bfea99df5d32e20b50d33838d0f
999	4b076d6feb5adfcf578ba183bdfe1dda
1000	0cf8e44a9dfe0e703c508bf1da4eb4b1
1001	sjdlfjsdklfjsdlkjflksdjf

(6 строк)

```
goodgoldlake=# insert into logs111 (id, entry) values (1002,'fdsjkhjksdhfjkdhsjkjlfhskjdf');  
INSERT 0 1
```

```
goodgoldlake=# select * from logs111 offset 995;
```

id	entry
996	8c77ae1534a4ac4544404589693f3da1
997	0bb6a25c8d5e6753cf3267db7ea58fd5
998	98356bfea99df5d32e20b50d33838d0f
999	4b076d6feb5adfcf578ba183bdfe1dda
1000	0cf8e44a9dfe0e703c508bf1da4eb4b1
1001	sjdlfjsdklfjsdlkjflksdjf
1002	fdsjkhjksdhfjkdhsjkjlfhskjdf

(7 строк)

```
goodgoldlake=# select * from sales111 offset 995;
```

id	data
996	996
997	997
998	998
999	999
1000	1000
1001	
1002	
1003	1003

(8 строк)

```
goodgoldlake=# insert into sales111 (id, data) values (1004, 1004);  
INSERT 0 1
```

```
goodgoldlake=# select * from sales111 offset 995;
```

id	data
996	996
997	997
998	998
999	999
1000	1000
1001	
1002	
1003	1003
1004	1004

(9 строк)

```

goodgoldlake=# drop table sales111, logs111;
DROP TABLE
goodgoldlake=# select * from sales111;
ОШИБКА:  отношение "sales111" не существует
СТРОКА 1: select * from sales111;
                        ^

goodgoldlake=# select * from logs111;
ОШИБКА:  отношение "logs111" не существует
СТРОКА 1: select * from logs111;
                        ^

```

на резервном узле

```
pg_dump -p 9425 -d goodgoldlake -f /var/db/postgres3/dump_result.sql
```

```

--
-- PostgreSQL database dump complete
--

```

на основном узле

```
scp postgres3@pg106:/var/db/postgres3/dump_result.sql /var/db/postgres1/
```

```

psql:/var/db/postgres1/dump_result.sql:111: ОШИБКА:  отношение "reports111" уже существует
ALTER TABLE
psql:/var/db/postgres1/dump_result.sql:126: ОШИБКА:  отношение "reports111_id_seq" уже существует
ALTER SEQUENCE
ALTER SEQUENCE
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
SET

```

```
goodgoldlake=# select * from sales111 offset 995;
```

id	data
996	996
997	997
998	998
999	999
1000	1000

(5 строк)

```
goodgoldlake=# select * from logs111 offset 995;
```

id	entry
996	8c77ae1534a4ac4544404589693f3da1
997	0bb6a25c8d5e6753cf3267db7ea58fd5
998	98356bfea99df5d32e20b50d33838d0f
999	4b076d6feb5adfcf578ba183bdfe1dda
1000	0cf8e44a9dfe0e703c508bf1da4eb4b1

(5 строк)

Выводы

- Настроено непрерывное резервное копирование с использованием WAL.
- Реализованы сценарии восстановления при различных типах сбоев.
- Показана эффективность использования pg_basebackup и архивирования WAL.
- Логическое восстановление через pg_dump подтвердило возможность точечного восстановления данных.